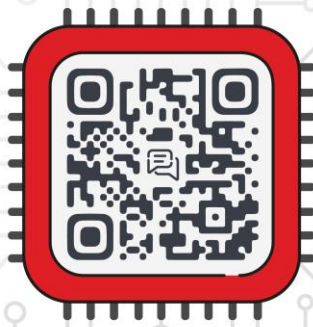




1

14

1680 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Intro to Trees

د. مدحت الصوص



RB Informatics; 4/28/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

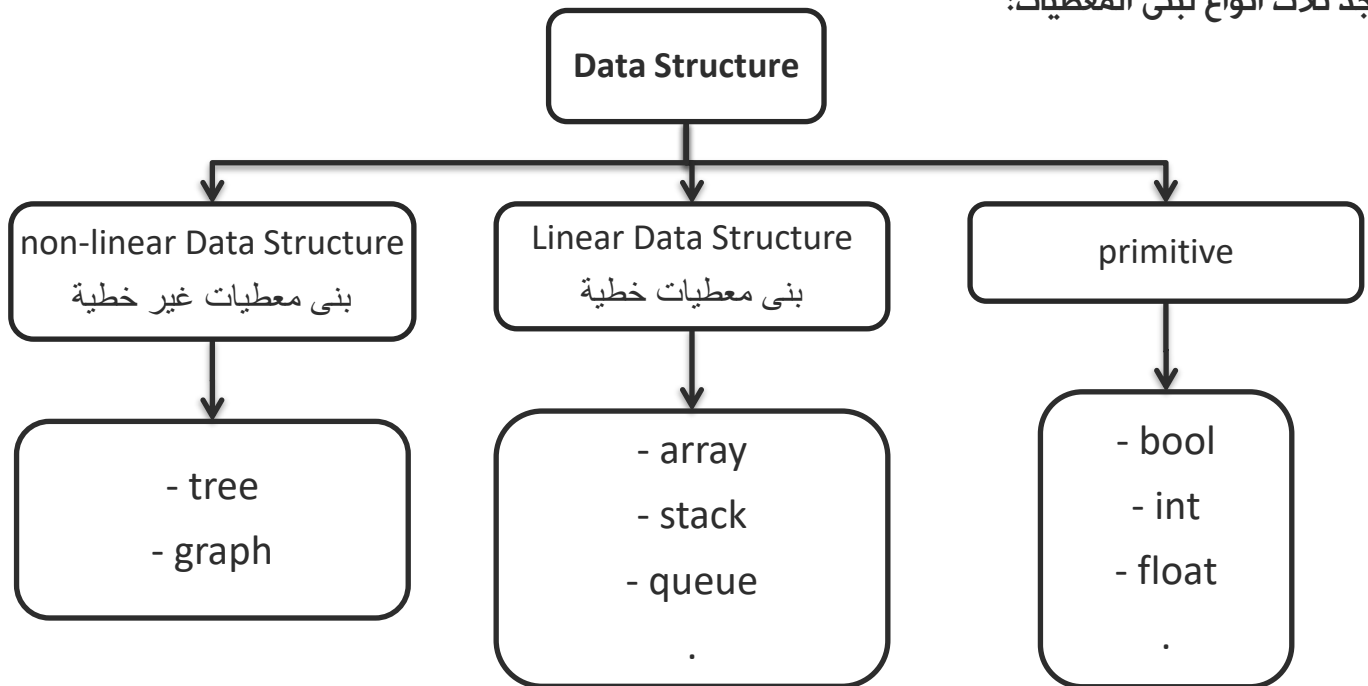
مرحباً أصدقائي 😊، عدنا لكم من جديد بمادة الخوارزميات-2، حيث تعتبر هذه المادة هي المادة الوحيدة فهذا الفصل التي لها علاقة بالبرمجة، لذلك يجب علينا التركيز عليها وفهمها جيداً.

■ ملاحظة: سوف تقتصر مواضيع الخوارزميات-2 إلى الـ tree و graph وبعض الخوارزميات المرفقة معها.

■ المرجع المستخدم لهذه المادة هو: (introduction to algorithm)

🔖 مقدمة:

يوجد ثلاث أنواع لبنى المعطيات:



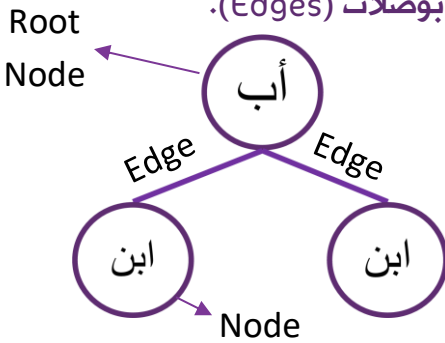
■ ملاحظة: سميت بنى خطية لأن العناصر تكون مرتبة في تسلسل محدد (كل عنصر مرتبط بعنصر سابق أو بعنصر لاحق مباشر) مثل array و الـ queue. وسميت بنى غير خطية لأن العناصر ليس لها ترتيب تسلسلي ثابت وقد يرتبط العنصر بعدة عناصر أخرى.

■ ملاحظة مهمة: الخوارزميات لا تحفظ وإنما تفهم ويجب فهم خطوات تطبيق الخوارزمية لاستخدامها في مشاكل مختلفة.

الشجرة Tree

ما هي ال Tree؟

- مجموعة من العناصر (items) نسميها عقد (Nodes)، مرتبطة مع بعضها البعض بوصلات (Edges).
- حيث يوجد قيود معينة عليها لكي نطلق عليها اسم شجرة:
- يوجد نوع واحد من الوصلات (من الأب إلى الابن).
 - كل عقدة لها أب ما عدا عقدة واحدة تكون بلا أب تسمى الجذر (root) وهي التي تبدأ منها الشجرة.
 - كل ابن له أب وحيد، وكل أب له ابن واحد أو عدة أبناء أو ليس له أي ابن.
 - تحمل ال Data على العقد، بينما الوصلات لا تحمل Data.
- ملاحظة: يمكن أن تحمل الوصلات Data، ولكن لن نتعامل معها في دراستنا.

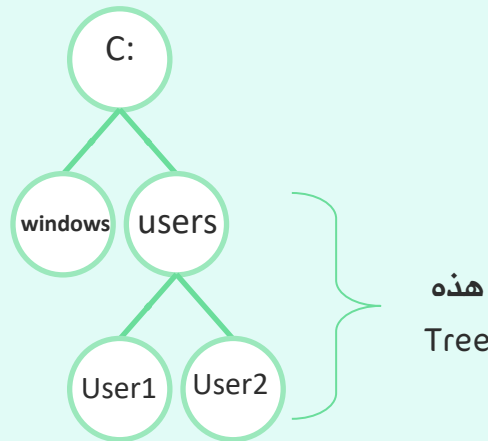
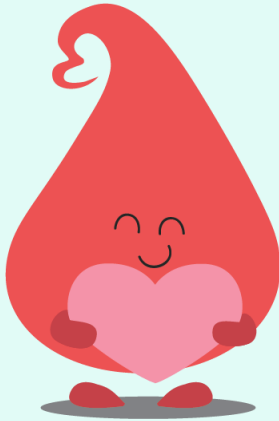


أمثلة عن استخدام ال Tree؟

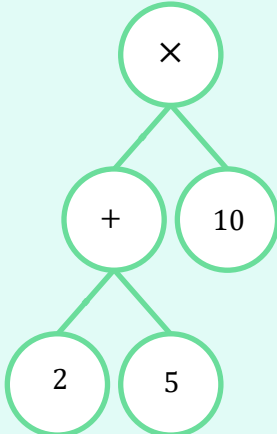
تستخدم ال Tree في شجرة العائلة أو الوراثة في ال oop (وهذا في اللغات التي تسمح بالوراثة من أب واحد فقط مثل ال Java) أو الملفات في الحاسب.

مثال:

- في ملفات الحاسب عند الدخول الى القرص c: سيظهر لنا الملفات في داخل القرص (الأبناء) وكل ملف يحتوي أيضاً على ملفات أبناء له أيضاً



- يمكن استخدام الأشجار لتمثيل معادلات رياضية، فنحل المعادلات الأبناء أولاً ثم إلى الآباء حتى نصل للجذر.



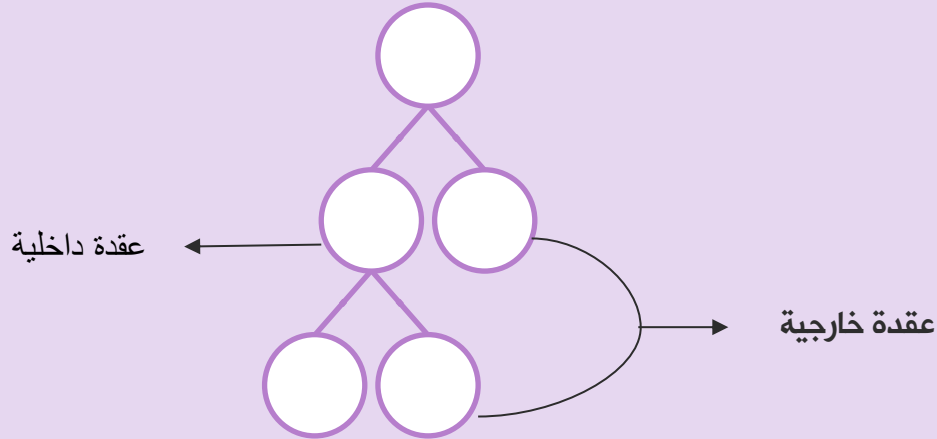
حيث الشجرة السابقة تمثل المعادلة :

$$(2 + 5) \times 10$$

تعريف هامة

1. أنواع العقد:

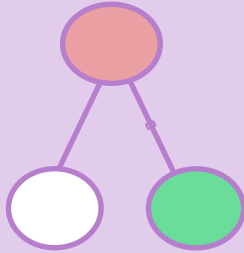
- عقد داخلية (Internal Nodes): هي أي عقدة تحتوي على ابن أو أكثر.
 - عقد خارجية (External Nodes): هي عقدة ليس لها أبناء وتسمى أيضاً ب **الورقة (leaf)**.
- ملاحظة: ليس من الضروري أن تكون العقدة الخارجية في أدنى مستوى للشجرة وكيفي أن تكون بلا أبناء.



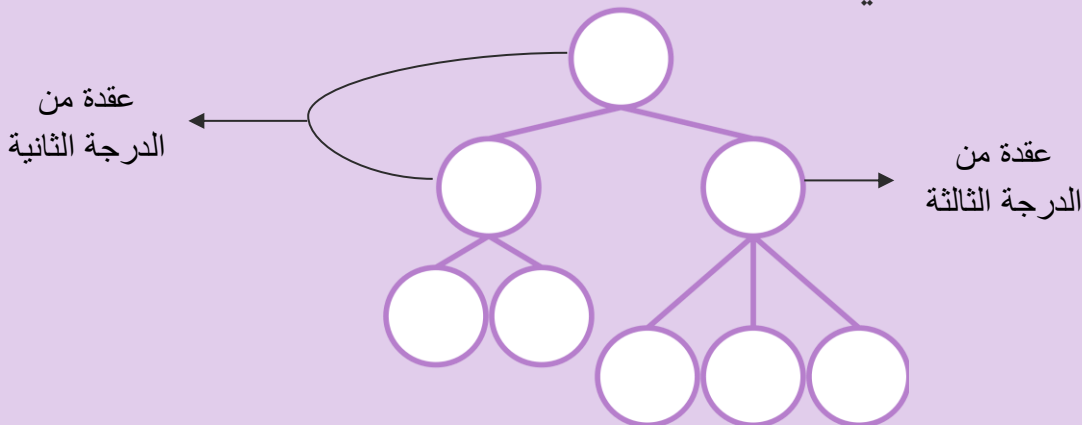
2. درجة العقدة: هو عدد أبناء هذه العقدة.

■ عقدة من الدرجة الثانية (لها ابناء).

■ عقدة من الدرجة صفر (ليس لها أبناء).



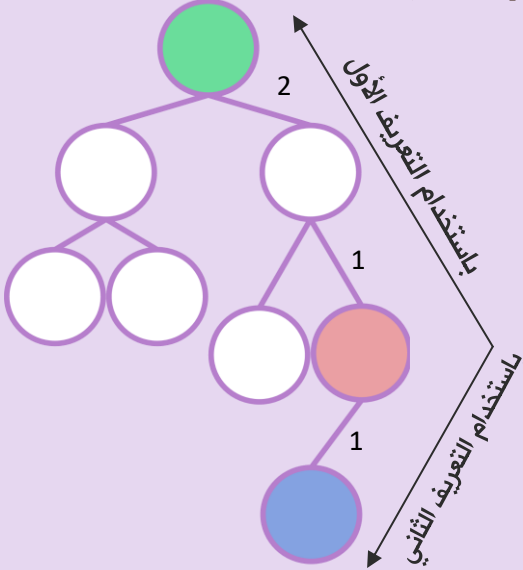
3. درجة الشجرة: هي أعلى درجة عقدة في هذه الشجرة.



نستنتج أن هذه الشجرة من الدرجة الثالثة لأن أعلى درجة عقدة فيها هي من الدرجة الثالثة.

4. ارتفاع العقدة: ولها تعريفان:

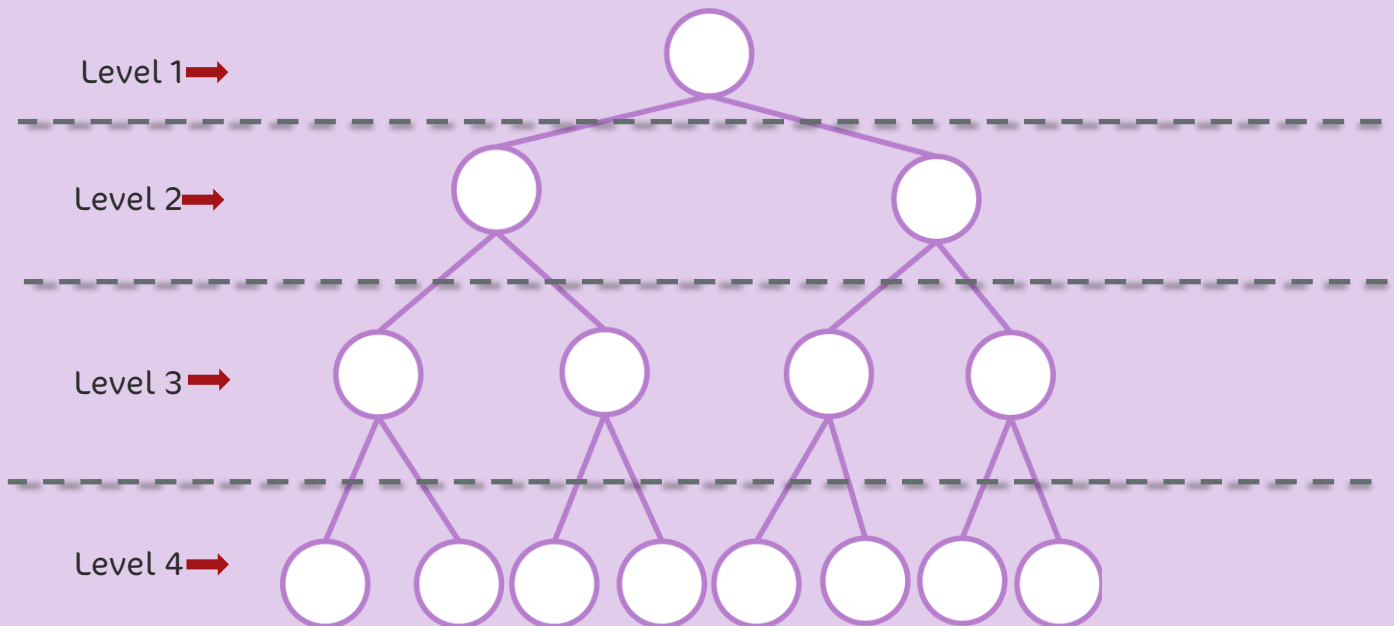
- التعريف الأول: ارتفاع العقدة هو عدد الوصلات من العقدة إلى الجذر.
- التعريف الثاني: ارتفاع العقدة هو عدد الوصلات من العقدة إلى آخر ورقة لهذه العقدة.



- باستخدام التعريف الأول فإن العقدة ارتفاعها هو (2) لأنه يوجد وصلتان بينها وبين الجذر.
- أما باستخدام التعريف الثاني فإن العقدة ارتفاعها هو (1) لأنه يوجد وصلة واحدة بين العقدة وآخر ورقة من الأسفل.

5. ارتفاع الشجرة: هو أعلى ارتفاع عقدة في الشجرة، وإذا تم استخدام أحد التعريفين الأول أو الثاني فسوف يكون ارتفاع الشجرة ذاته في كلا التعريفين.
- فبالمثال السابق سيظهر لنا أن ارتفاع الشجرة هو (3) سواء استخدمنا التعريف الأول أو الثاني. كالتالي:
- باستخدام التعريف الثاني فإن العقدة ارتفاعها هو (3) لأنه يوجد (3) وصلات بينها وبين الجذر.
- باستخدام التعريف الأول فإن العقدة ارتفاعها هو (3) لأنه يوجد (3) وصلات بينها وبين الجذر.

6. مستوى الشجرة: هو عدد الطبقات في الشجرة.

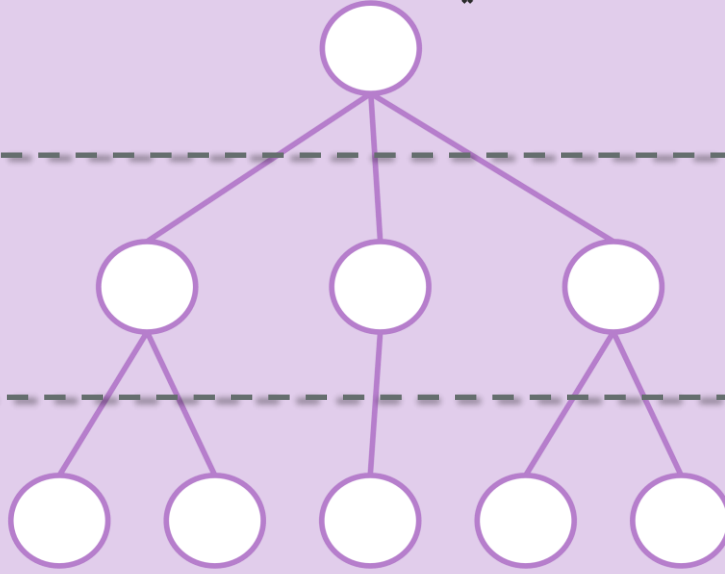


7. عرض الشجرة: هو أكبر عدد للعقد في المستوى الواحد للشجرة.

→ العرض (1) لوجود عقدة واحدة

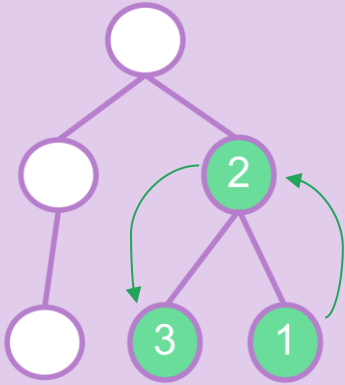
→ العرض (3) لوجود ثلاث عقد

→ العرض (5) لوجود خمس عقد



ومنه فإن عرض هذه الشجرة هو (5) لأن المستوى الثالث للشجرة يحتوي على خمس عقد.

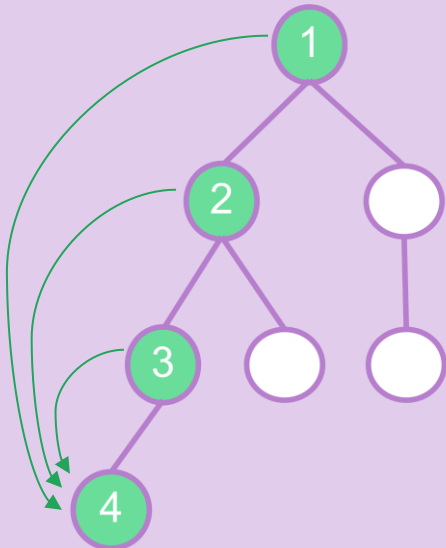
8. المسار (path): هو تتالي العقد، أي: الطريق الذي يصل عقدة بعقدة أخرى، ولكن بشرط وجود وصلات فيما بينهم.



- الطريق من العقدة (1) إلى العقدة (3) يسمى مسار (path).
- ولكن إذا لم يكن هناك وصلة بين العقد فهذا لا يعتبر مسار.

9. حجم الشجرة: هو عدد العقد في الشجرة.

مثلاً الشجرة السابقة حجمها هو (6) لوجود ستة عقد فيها.



10. السلف (Ancestor): هو أي عقدة تقع على المسار من العقدة

الحالية صعوداً نحو الجذر (Root).

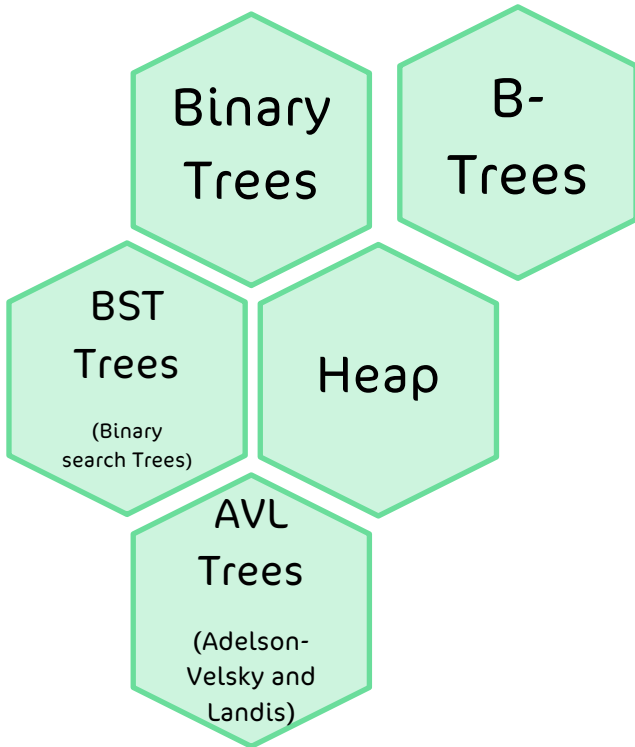
- العقد (1) و (2) و (3) و (4) هي سلف للعقدة (4).

- أي: كل ما سبق العقدة من آباء يعتبر سلف لهذه العقدة بالإضافة للعقدة نفسها تعتبر سلف ancestor لنفسها.

ملاحظة أرييسيزية:

Proper Ancestor: هو السلف باستثناء العقدة الحالية.

أنواع ال Trees



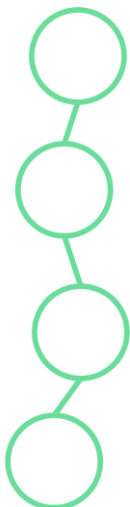
الأشجار الثنائية Binary Trees

- هي حالة خاصة من الأشجار العادية، ولكن الفرق أننا نضيف شرط على هذه الشجرة:
- كل عقدة لها ابنان **على الأكثر** فقط، أي إما ابن واحد أو ابنان أو ولا ابن، يدعى الابن الأيسر ب ال (Left-Child Node) و الابن الأيمن ب (Right-Child Node).

ملاحظة:

ليس من الضروري أن تكون درجة الشجرة الثنائية هي (2)، ولكن من الخطأ أن تكون أكبر من (2).

مثلاً



الشجرة المجاورة تعتبر شجرة ثنائية لأن كل عقدة تحتوي على ابن واحد فقط أو ولا ابن أي الشجرة من الدرجة الأولى. ولكن إذا كانت أحد هذه العقد تحتوي على أكثر من ابنين فهذه ليست شجرة ثنائية.

إثراء:

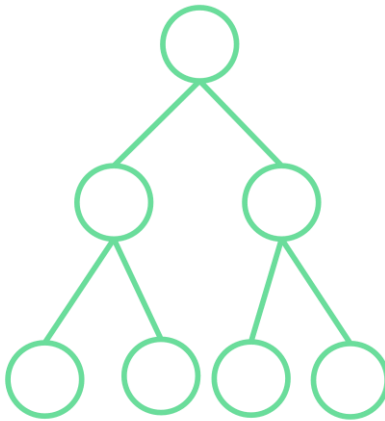
أسلوب التعامل مع الشجرة الثنائية أبسط من الشجرة العادية، لأننا نتعامل مع left & right أي (ابن يميني وابن يساري) بدلاً من التعامل مع عدد غير معلوم من الأبناء.

بعض أنواع ال Binary Trees

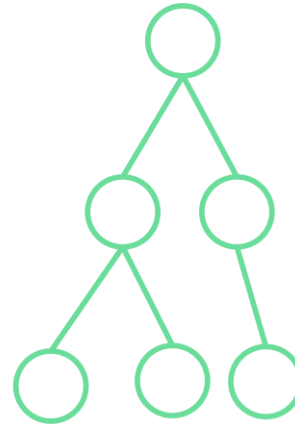
1. Full Binary Tree

كل عقدة في هذه الشجرة تكون من الدرجة الثانية (أي لها ابنان "باستثناء الأوراق")، وبالتالي كل المستويات في الشجرة يجب أن تحتوي على أقصى عدد ممكن من العقد، ضمن حدود الشجرة الثنائية.

Full Tree ✓



Not A Full Tree ✗

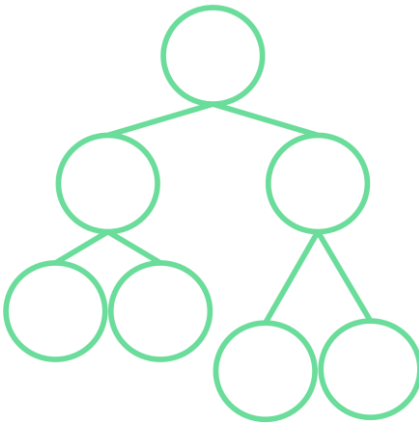


ملاحظة آريسية:

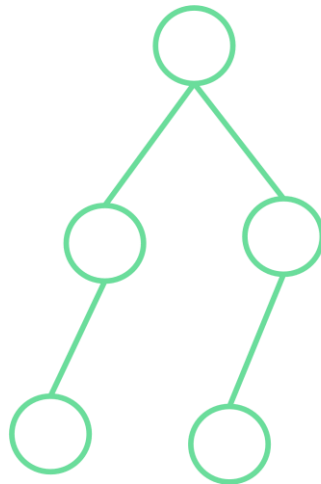
نلاحظ أن أقصى عدد من العقد في المستوى الواحد للشجرة هو $2^{level-1}$ مثلاً: عدد العقد في المستوى الثالث هو: $2^3-1 = 2^2 = 4$ وهو أقصى عدد عقد ممكن أن يكون في المستوى الثالث.

2. Complete Tree

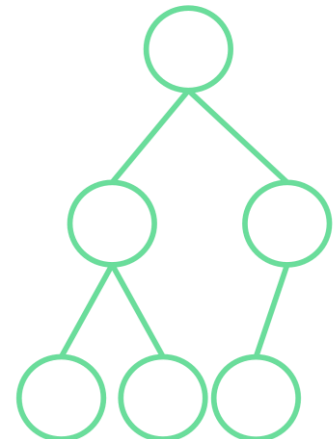
هي شجرة فيها كل مستوى ممتلئ (Full) ما عدا **المستوى الأخير** يمكن أن يحوي فراغات، ولكن يشترط بهذه الفراغات أن تكون **من اليمين باتجاه اليسار حصراً**.



Complete Tree ✓



Not A Complete Tree ✗



Complete Tree ✓

شرح:

- الشجرة على اليمين هي complete tree لأنه ينقصها عقدة في المستوى الأخير ومن اليمين.
- الشجرة في المنتصف لا تعتبر complete tree لأنه ينقصها عقدة من اليسار في المستوى الأخير.
- ال full tree هي حالة خاصة من complete tree، أي كل full tree هي حتماً complete tree ولكن العكس غير صحيح، وبالتالي الشجرة على اليسار هي complete tree.
- ملاحظة: حتى لو حذفنا جميع العقد من المستوى الأخير فلا بأس، ولكن الخطأ أن نحذف من اليسار أو من مستويات قبل المستوى الأخير.
- ملاحظة: المفهومين السابقين (Full & Complete Trees) ينطبقان على جميع الأشجار محدودة الدرجة مثل الأشجار الثنائية والثلاثية و ، لكن لا يمكننا تطبيقها على الأشجار العادية لأنها ليست محدودة الأبناء.

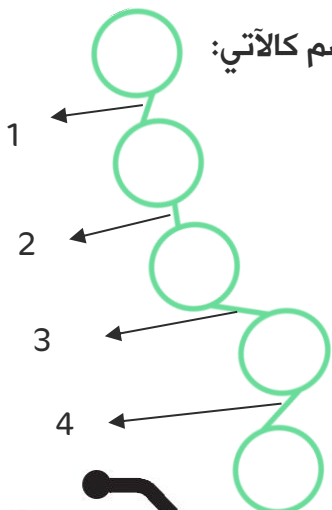
■ **سؤال:** لو كان لدينا شجرة ثنائية مؤلفة من n عقدة، فهل يمكن أن نعرف شكل هذه الشجرة؟ لنفرض لدينا شجرة مكونة من (5) عقد، فيمكن أن يكون شكلها:



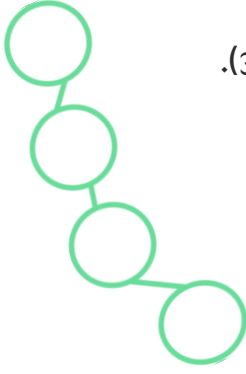
نلاحظ: أنه من الممكن أن نوجد العديد من الأشكال لهذه الشجرة، ولكن ماذا لو كنا نريد أن نجد أكبر ارتفاع ممكن لهذه الشجرة أو إيجاد أصغر ارتفاع ممكن لهذه الشجرة أيضاً؟

✍ إيجاد أكبر ارتفاع ممكن للشجرة:

- في المثال السابق: لو أردنا إيجاد أكبر ارتفاع لشجرة ثنائية مكونة من (5) عقد، فإنها ترسم كالآتي: نلاحظ أن أعلى ارتفاع ممكن لتشكيل شجرة ثنائية مكونة من (5) عقد هو (4).



- **تذكر:** ارتفاع الشجرة الثنائية هو أكبر ارتفاع عقدة فيها، أي هو عدد الوصلات بين الجذر (root) وبين آخر ورقة (leaf) في المستوى الأخير.



- ولو كانت الشجرة الثنائية مكونة من (4) عقد، فيكون أكبر ارتفاع للشجرة هو (3).
- نستنتج أن:
- أعلى ارتفاع لشجرة ثنائية مكونة من (n) عقدة هو: $n - 1$.
- ومنه فإن ارتفاع الشجرة الثنائية H يكون: $H \leq n - 1$.

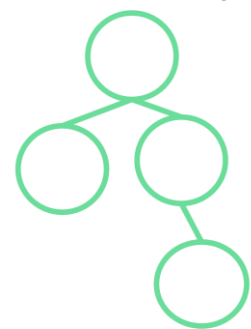
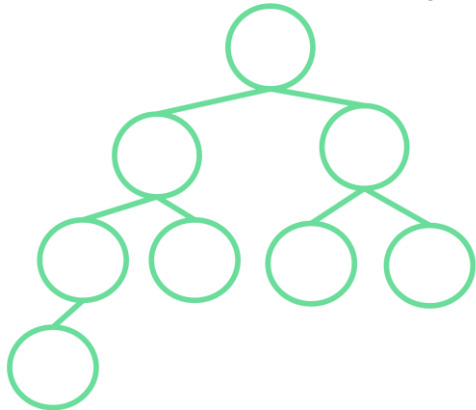
✍ إيجاد أصغر ارتفاع ممكن للشجرة:

ليكن لدينا الجدول الآتي، ولنحاول استنتاج قانون أصغر ارتفاع ممكن:

عدد العقد (n)	الارتفاع الأقل	$\log_2 n$
4	2	2
5	2	2.32
6	2	2.58
7	2	2.8
8	3	3
.	.	.
.	.	.
.	.	.
15	3	3.9
16	4	4

من الجدول نجد:

- لو كان لدينا (4) عقد مثلاً، سنجد أن أقل ارتفاع ممكن للشجرة هو (2).
- لو كان لدينا (6) عقد مثلاً، سنجد أن أقل ارتفاع ممكن للشجرة هو (2).
- لو كان لدينا (8) عقد مثلاً، سنجد أن أقل ارتفاع ممكن للشجرة هو (3).



ومنه فإننا كلما نصل لعدد من قوى العدد (2)، نقوم بزيادة ارتفاع الشجرة بمقدار (1).

- لذلك فإن أقل ارتفاع للشجرة يمثل ب: $\log_2 n$

- إذا كان ناتج $\log_2(n)$ يحتوي على فواصل، نأخذ القسم الصحيح ونهمل القسم العشري.
- **مثال:** $\log_2 5 = 2.32$ أي أقل ارتفاع لشجرة مكونة من (5) عقد هو (2).
- **ملاحظة:** يوجد تابع يقوم بحذف القسم العشري في البرمجة يسمى تابع ال floor() .

استنتاج:

إن ارتفاع أي شجرة ثنائية (Binary Tree) مكونة من (n) عقدة يكون:

$$\text{أقل ارتفاع} \leftarrow \log_2(n) \leq H \leq n - 1 \rightarrow \text{أكبر ارتفاع}$$

وهذا حصراً في حالة الشجرة الثنائية.

Binary Trees Representation

By

Pointers

By

One-Dimensional Array

By

Two-Dimensional Array



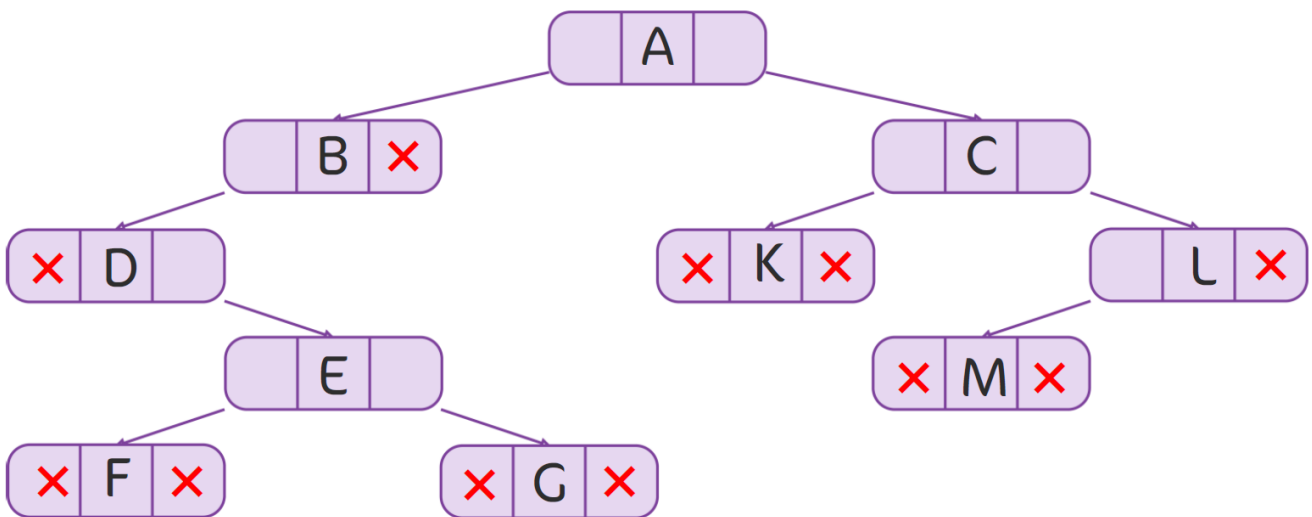
❧ كيف يمكننا تمثيل الشجرة الثنائية برمجياً؟

هناك عدة حلول لتمثيلها:

1. استخدام المؤشرات Pointers:

أي كل عقدة تحتوي على Data (قيمة العقدة) ومؤشرين (tow pointer) للابن اليساري والابن اليميني.
 - في هذه الطريقة يمكن معرفة العقدة الابن، ولكن لا يمكن معرفة العقدة الأب، لعدم وجود مؤشر (pointer) يُوَشر على الأب.
 ملاحظة: يمكن إضافة مؤشر آخر للعقدة ليُوَشر على الأب.
 ملاحظة: في هذا الحل (pointers) كل الشجرة تتعلق بالجزر (root)، فإذا فقدنا الجذر فسوف نفقد الشجرة بأكملها.

مثال لتمثيل شجرة ثنائية باستخدام المؤشرات:



2. تخزين الشجرة الثنائية بمصفوفة ثنائية:

في هذه الطريقة لا نقوم بتخزين شكل الشجرة الاعتيادي، وإنما نخزن ال Data التي فيها.

- طريقة التخزين:

1. نكون مصفوفة ثنائية مكونة من n سطر لتخزين شجرة مكونة من n عقدة.

2. نشكل ثلاث أعمدة:

- عامود أول يكون ل Data.

- عامود ثاني يكون ل index الخاص بالابن اليساري.

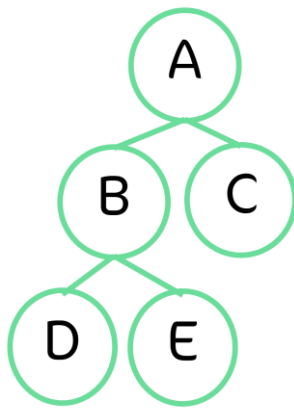
- عامود ثالث يكون ل index الخاص بالابن اليميني.

3. نضع الجذر في أول سطر وفي أول عامود (أي نضع ال Data المخزنة فيه).

4. في العامود الثاني والثالث نضع ال index الخاص بالابن اليساري و ال index الخاص بالابن اليميني ل Data الموجودة في العامود الأول.

5. إذا لم يكن يوجد أبناء للعقدة فإننا نضع (-1) مكان ال index.

مثال: لو أردنا تخزين الشجرة المجاورة بمصفوفة ثنائية:



نلاحظ أن الشجرة مكونة من (5) عناصر وبالتالي المصفوفة الثنائية ستكون مؤلفة من (5) أسطر.

- بدأنا من الجذر وهو ال (A) في السطر الأول والعمود الأول في المصفوفة.

- قمنا بتخزين ال index الخاص بالابن اليميني

والابن اليساري لكل عقدة،

مثلاً: العقدة (B) لها ابن يساري (D) وابن

يميني (E)

العقدة (D) موجودة بال index رقم (3)، لذلك

وضعنا الرقم (3) للابن اليساري للعقدة (B).

وكذلك نفس الشيء مع العقدة (E) هي

موجودة بال index رقم (4) فالابن اليميني

للعقدة (B) سيكون بال index رقم (4).

- العقد (C) و (E) و (D) لا تحتوي على أبناء لذلك وضعنا (-1) عند ال index الخاص بأبنائها.

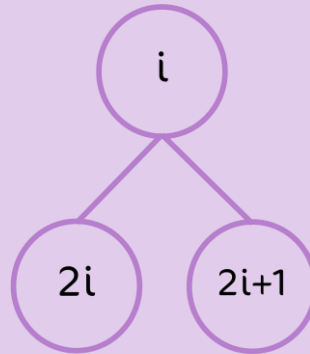
Index	Data	Left	Right
0	A	1	2
1	B	3	4
2	C	-1	-1
3	D	-1	-1
4	E	-1	-1

■ ملاحظة:

- ترتيب ال Data في الأسطر غير مهم، أي من الممكن أن تكون العقدة (B) موجودة بالسطر الأخير والعقدة (E) موجودة بالسطر الثاني
- ولكن من الضروري أن يكون **الجذر بالسطر الأول**.
- نلاحظ أن المصفوفة الثنائية السابقة تحتوي على نوعين من المتغيرات (char & int) لذلك يمكننا قسم المصفوفة إلى مصفوفتين، واحدة تكون ل Data واحدة تكون ل index ولكن مع المحافظة على نفس الأرقام لكل ابن يميني ويساري.

3. تخزين الشجرة الثنائية بمصفوفة أحادية:

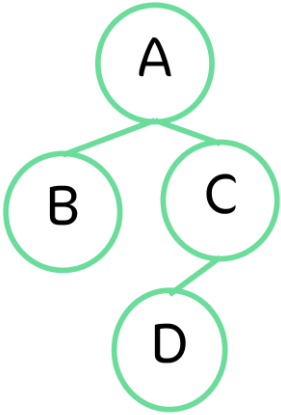
1. نقوم بحجز مصفوفة حجمها يساوي: 2^{level}
2. نضع الجذر في ال index رقم (1).
3. لتخزين الابن اليميني واليساري نستخدم هذه الطريقة:
إذا كان الأب في ال index رقم (i) فالابن اليساري يكون بال index رقم ($2i$) والابن اليميني يكون بال index رقم ($2i + 1$).



4. إذا لم يكن للعقدة أبناء فإن مكان ابنائها يبقى محجوزاً ولا يتم أخذه من قبل عقدة أخرى.



مثال: تخزين الشجرة الثنائية الآتية بمصفوفة أحادية بالشكل الآتي:



x	A	B	C	x	x	D	x
0	1	2	3	4	5	6	7

➤ شرح:

■ حجزنا مصفوفة حجمها: $2^{\text{level}} = 2^3 = 8$

■ نترك ال index رقم (0) فارغ.

■ قمنا بتخزين الجذر (A) في ال index رقم (1).

ولديه الابن اليساري (B) الذي سيوضع بال index رقم $2i = 2(1) = 2$

ولديه الابن اليميني (C) الذي سيوضع بال index رقم $2i + 1 = 2(1) + 1 = 3$

■ وكذلك نفس الشيء مع العقدة (C) لديها الابن اليساري (D) الذي سيوضع بال index رقم $2(3) = 6$

■ كل الأماكن التي لا يوجد بها Data تبقى محجوزة وتحمل القيمة null.

■ فمثلاً لو أردنا إضافة ابن للعقدة (B) فإننا نضع ال Data الخاصة به في المكان المحجوز لأبناء العقدة (B)

■ ملاحظة:

هذا الحل يأخذ الكثير من المساحة لوجود الكثير من الخانات الفارغة بالمصفوفة، ولكن عندما تكون الشجرة Full فيعتبر هذا الحل مناسب لأنه يملأ جميع الفراغات في المصفوفة.



THE END ^ _ ^